

List Comprehensions		
squares = [x * x x <- [1..]]		
Declarando Types e Classes		
type synonym type	MyType = Type type PairList a b = [(a,b)] type String = [Char] -- from Prelude	
data (single constructor)	data MyData = MyData Type Type deriving (Class, Class)	
data (multi constructor)	data MyData = Simple Type Duple Type Type Nople	
data (record syntax)	data MDt = MDt { fieldA , fieldB:: TyAB , fieldC:: TyC }	
newtype	newtype MyType = MyType Type (single constr./field) deriving (Class,Class)	
typeclass	class MyClass a where foo:: a -> a -> b goo:: a -> a	
typeclass in-stance	instance MyClass MyType where foo x y = ... goo x = ...	

Misc		
id	:: a -> a	id x = x -- identity
const	:: a -> b -> a	(const x) y = xc
undefined	:: a	undefined = ⊥ (lifts error)
error	:: [Char] -> a	error cs = ⊥ (lifts error cs)
not	:: Bool -> Bool	not True = False
flip	:: (a->b->c)->b->a->c	flip f \$ x y = f y x

Listas		
null	:: [a] -> Bool	null [] = True -- ∅?
length	:: [a] -> Int	length [x,y,z] = 3
elem	:: a -> [a] -> Bool	y ‘elem’ [x,y] = True -- ∈?
head	:: [a] -> a	head [x,y,z,w] = x
last	:: [a] -> a	last [x,y,z,w] = w
tail	:: [a] -> [a]	tail [x,y,z,w] = [y,z,w]
init	:: [a] -> [a]	init [x,y,z,w] = [x,y,z]
reverse	:: [a] -> [a]	reverse [x,y,z] = [z,y,x]
take	:: Int -> [a] -> [a]	take 2 [x,y,z] = [x,y]
drop	:: Int -> [a] -> [a]	drop 2 [x,y,z] = [z]
takeWhile, dropWhile	:: (a -> Bool) -> [a] -> [a]	takeWhile (/= z) [x,y,z,w] = [x,y]

zip	:: [a] -> [b] -> [(a, b)]	zip [x,y,z] [a,b] = [(x,a), (y,b)]
repeat	:: a -> [a]	repeat x = [x,x,x,x,x,x,...]
cycle	:: [a] -> [a]	cycle xs = xs ++xs ++xs ++... cycle [x,y] = [x,y,x,y,x,y,...]
iterate	:: (a -> a) -> a -> [a]	iterate f x = [x,f x,f (f x),...]

Higher-order / Functors	
map	:: (a->b) -> [a] -> [b]
map	f [x,y,z] = [f x, f y, f z]
zipWith	:: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith	f [x,y,z] [a,b] = [f x a, f y b]
filter	:: (a -> Bool) -> [a] -> [a]
filter	(/=y) [x,y,z] = [x,z]
foldr	:: (a -> b -> b) -> b -> [a] -> b
foldr	f z [x,y] = x ‘f’ (y ‘f’ z)
foldl	:: (a -> b -> a) -> a -> [b] -> a
foldl	f x [y,z] = (x ‘f’ y) ‘f’ z

Special folds	
and	:: [Bool] -> Bool and [p,q,r] = p && q && r
or	:: [Bool] -> Bool or [p,q,r] = p q r
sum	:: Num a => [a] -> a sum [i,j,k] = i + j + k
product	:: Num a => [a] -> a product [i,j,k] = i *j *k
maximum	:: Ord a => [a] -> a maximum [9,0,5] = 9
minimum	:: Ord a => [a] -> a minimum [9,0,5] = 0
concat	:: [[a]] ->[a] concat [xs,ys,zs] = xs ++ys ++zs

```
-- Tipo Categoria
data Categoria = Livro | Brinquedo | Escritorio
deriving (Show, Eq)
-- Tipo Produto
data Produto = Produto {
  nome :: String,
  valor :: Double,
  categoria :: Categoria
} deriving (Show, Eq)
-- Lista de exemplos
produtos :: [Produto]
produtos = [
  Produto "Livro A" 50.0 Livro,
  Produto "Carrinho" 30.0 Brinquedo,
  Produto "Caderno" 20.0 Escritorio,
  Produto "Livro B" 40.0 Livro,
  Produto "Boneca" 25.0 Brinquedo
```

```
]
-- 1. extrair: retorna lista de preços
extrair :: [Produto] -> [Double]
extrair ps = [valor p | p <- ps]
-- 2. minValor: menor preço
minValor :: [Produto] -> Double
minValor ps = minimum (extrair ps)
-- 3. livrEscr: remove brinquedos
livrEscr :: [Produto] -> [Produto]
livrEscr ps = [p | p <- ps, categoria p /= Brinquedo]
-- 4. avgLivrEscr: média dos não brinquedos
avgLivrEscr :: [Produto] -> Double
avgLivrEscr ps =
  let lista = extrair (livrEscr ps)
  in sum lista / fromIntegral (length lista)
-- 5. maxMinLivr: maior e menor preço dos livros
maxMinLivr :: [Produto] -> (Double, Double)
maxMinLivr ps =
  let livros = [valor p | p <- ps, categoria p == Livro]
  in (maximum livros, minimum livros)
-- 6. countBrinq: quantidade de brinquedos
countBrinq :: [Produto] -> Int
countBrinq ps = length [p | p <- ps, categoria p == Brinquedo]
```

Versão Folder

```
-- 1. extrair: usando map
extrair :: [Produto] -> [Double]
extrair = map valor
-- 2. minValor: usando fold
minValor :: [Produto] -> Double
minValor ps = foldr1 min (map valor ps)
-- 3. livrEscr: usando filter
livrEscr :: [Produto] -> [Produto]
livrEscr = filter (\p -> categoria p /= Brinquedo)
-- 4. avgLivrEscr: usando map, filter e fold
avgLivrEscr :: [Produto] -> Double
avgLivrEscr ps =
  let lista = map valor (filter (\p -> categoria p /= Brinquedo) ps)
  (soma, qtd) = foldr (\x (s, c) -> (s + x, c + 1)) (0, 0) lista
  in soma / fromIntegral qtd
-- 5. maxMinLivr: usando filter, map e fold
maxMinLivr :: [Produto] -> (Double, Double)
maxMinLivr ps =
  let livros = map valor (filter (\p -> categoria p == Livro) ps)
  in foldr1 (\x (mx, mn) -> (max x mx, min x mn))
    (map (\v -> (v, v)) livros)
-- 6. countBrinq: usando filter
countBrinq :: [Produto] -> Int
countBrinq = foldr (\p acc -> if categoria p == Brinquedo then acc + 1 else acc) 0
```

Questão 2

Considere o tipo data Sozinho = Sozinho, crie uma instância de Monoid para este tipo.

```
data Sozinho = Sozinho -- único valor possível monóide trivial
instance Semigroup Sozinho where
  Sozinho <> Sozinho = Sozinho
instance Monoid Sozinho where
  mempty = Sozinho
Considere o operador binário:
```

`ign :: String -> String -> String`
`ign l m = m`
O tipo `String` com esta operação `ign` e o elemento neutro `[]` formam um monóide? Justifique sua resposta. Queremos saber se `(String, ign, [])` forma um monóide.

Associatividade ✓

```
ign x (ign y z) == ign (ign x y) z
-- Lados esquerdo
ign x (ign y z) = ign x z = z
-- Lado direito
ign (ign x y) z = ign y z = z
```

Elemento neutro ✗

Para ser monóide, `[]` deve satisfazer:
`ign [] x = x` -- ✓
`ign x [] = []` -- ✗ (deveria ser `x`)
Logo, não satisfaz a definição de monóide. Operações como: `ign l m = m` (pega o último) `op l m = l` (pega o primeiro) não são monóides sozinhas, porque: ➡ não possuem elemento neutro válido Mas podem virar monóides quando encapsuladas em tipos como: `Maybe First Last`

Questão 3

Considere o tipo `data Dupla a = Dupla a [Int]`

```
--Qual o kind de Dupla Bool?
Dupla Bool :: *
--Crie uma instância de Functor para Dupla.
instance Functor Dupla where
    fmap f (Dupla x xs) = Dupla (f x) xs
    --Apenas o primeiro campo é transformado (`a`).
    --A lista `[Int]` permanece igual.
--Qual o tipo da expressão Dupla '5' [0,1]?
Dupla Char
--Qual o tipo da expressão Dupla?
Dupla :: a -> [Int] -> Dupla a -- é um construtor de dados:
--Crie uma instância de Show que mostre na tela uma dupla em formato de tuplas do Haskell. Por exemplo, Dupla 'k' [1,2,3] deverá ser mostrado (k,[1,2,3]).
instance Show a => Show (Dupla a) where
    show (Dupla x xs) = "(" ++ show x ++ "," ++ show xs ++ ")"
--Faça uma função mostra :: Dupla a -> Either [Int] a que mostra a lista de inteiros caso o seu tamanho seja maior que zero ou o campo de tipo a caso contrário.
mostra :: Dupla a -> Either [Int] a
mostra (Dupla x xs)
    | null xs     = Right x
    | otherwise   = Left xs
```

Questão 4

```
--(a) \x -> x
a -> a --função identidade
--(b) id . tail $ "HELLO"
[Char] -- ou string, concreto não função
--(c) 4*9
`Num a => a` -- (*) é polimorfico typeclass Num
--(d) ("FATEC", False, 'K')
(String,Boll,Char)
--(e) [(False,False),(True,False),(False,True),(True,True)]
[(Bool,Bool)]
```

```
--(f) filter id
filter :: (a -> Bool) -> [a] -> [a]
id :: a -> a -- logo a = Bool
[Bool] -> [Bool]
```

Questão 5

Considere `data () = ()` e complete:

```
f1 :: (a,b) -> (b,a,a)
f1 (a,b) = (b,a,a)
f2 :: a -> (a,a,a,())
f2 a = (a,a,a,())
f3 :: Either a () -> Maybe a
f3 (Left a) = Just a
f3 (Right ()) = Nothing
f4 :: (a -> b) -> (b -> z) -> (a -> z)
f4 f g = g . f
f5 :: (a -> b) -> (c,a) -> (c,b)
f5 g = \ (c,a) -> (c, g a)
```

Pattern matching

```
factorial :: (Integral a) => a -> a
factorial 0 = 1
factorial n = n * factorial (n - 1)
head' :: [a] -> a
head' [] = error "lista vazia"
head' (x:_) = x
tell :: (Show a) => [a] -> String
tell [] = "A lista esta vazia"
tell (x:[]) = "um elemento"
tell (x:y:[]) = "dois elementos"
tell (x:y:_) = "MAIS de dois elementos"
capital :: String -> String
capital "" = "String vazia, oops!"
capital all@(x:xs) = "A primeira letra de " ++ all ++ " é " +
+ [x]
```

Guards

```
bmiTell :: (RealFloat a) => a -> a -> String
bmiTell weight height
    | bmi <= skinny = "Você esta abaixo do peso!"
    | bmi <= normal = "Supostamente você esta normal."
    | bmi <= fat = "Você esta gordo! Faça uma dieta."
    | otherwise = "Você é uma baleia, meus parabéns!"
where bmi = weight / height ^ 2
      skinny = 18.5
      normal = 25.0
      fat = 30.0
```

Let

```
cylinder :: (RealFloat a) => a -> a -> a
cylinder r h =
    let sideArea = 2 * pi * r * h
        topArea = pi * r ^2
    in sideArea + 2 * topArea
```

Recursividade

```
maximum' :: (Ord a) => [a] -> a
maximum' [] = error "maximum of empty list"
maximum' [x] = x
```

```
maximum' (x:xs) = max x (maximum' xs)
replicate' :: (Num i, Ord i) => i -> a -> [a]
replicate' n x
    | n <= 0 = []
    | otherwise = x:replicate' (n-1) x
take' :: (Num i, Ord i) => i -> [a] -> [a]
take' n _ | n <= 0 = []
take' _ [] = []
take' n (x:xs) = x : take' (n-1) xs
reverse' :: [a] -> [a]
reverse' [] = []
reverse' (x:xs) = reverse' xs ++ [x]
zip' :: [a] -> [b] -> [(a,b)]
zip' _ [] = []
zip' [] _ = []
zip' (x:xs) (y:ys) = (x,y):zip' xs ys
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x:xs) =
    let smallerSorted = quicksort [a | a <- xs, a <= x]
        biggerSorted = quicksort [a | a <- xs, a > x]
    in smallerSorted ++ [x] ++ biggerSorted
```

Lambda

```
numLongChains :: Int
numLongChains = length (filter (\xs -> length xs > 15) (map chain [1..100]))
flip' :: (a -> b -> c) -> b -> a -> c
flip' f = \x y -> f y x
```

Fold

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (\acc x -> acc + x) 0 xs
-- ou melhor
sum' :: (Num a) => [a] -> a
sum' = foldl (+) 0
maximum' :: (Ord a) => [a] -> a
maximum' = foldr1 (\x acc -> if x > acc then x else acc)
reverse' :: [a] -> [a]
reverse' = foldl (\acc x -> x : acc) []
product' :: (Num a) => [a] -> a
product' = foldr1 (*)
filter' :: (a -> Bool) -> [a] -> [a]
filter' p = foldr (\x acc -> if p x then x : acc else acc) []
head' :: [a] -> a
head' = foldr1 (\x _ -> x)
last' :: [a] -> a
last' = foldl1 (\_ x -> x)
map ($ 3) [(4+), (10*), (^2), sqrt]
>> [7.0,30.0,9.0,1.7320508075688772] -- resposta
```

Composição de Funções

```
(.) :: (b -> c) -> (a -> b) -> a -> c
f . g = \x -> f (g x)
map (negate . abs) [5,-3,-6,7,-3,2,-19,24]
>> [-5,-3,-6,-7,-3,-2,-19,-24]
map (negate . sum . tail) [[1..5],[3..6],[1..7]]
>> [-14,-15,-27]
oddSquareSum :: Integer
oddSquareSum = sum . takeWhile (<10000) . filter odd . map (^2) $ [1..]
```